

API and AJAX Integration Notes

API used

The website does not depend on a third-party public API. It uses its own Laravel web endpoints as the application API, with PostgreSQL/ Supabase supported in shared or production environments and SQLite supported locally. React pages communicate with Laravel through Inertia.js AJAX requests.

This design keeps credentials and database access on the server. The browser never connects directly to Supabase and no service-role key is exposed to JavaScript.

Purpose

The endpoints provide authentication, role-protected garden data, form validation, search/filtering, workflow mutations, notifications, reporting, and CSV export.

Main endpoints

Method and endpoint	Purpose	Data returned or changed
POST /login, POST /register, POST /logout	Authentication	Session and authenticated user

GET /garden-plots	Plot inventory and filtering	Paginated plot code, location, area, and status
POST /garden-plots, PUT /garden-plots/{id}	Plot management	Creates or updates a plot
GET /plot-requests	Request tracking	Paginated requests scoped by role
POST /plot-requests	Member request	Creates a pending request
POST /plot-requests/{id}/approve	Approval workflow	Approves request, creates assignment, occupies plot, sends notifications
POST /plot-requests/{id}/reject	Request decision	Rejects request with a reason and notification
GET /assignments	Assignment history	Paginated assignments scoped by role
POST /assignments, PUT /assignments/{id}	Assignment management	Creates an assignment or updates dates
GET /garden-calendar	Calendar content	Published events for members; all statuses for staff/admin
GET /community-updates	Announcement content	Published updates for members; all statuses for staff/admin
GET /reports	Operational reporting	Plot, request, assignment, and membership metrics
GET /reports/export	Report download	CSV request and assignment data

GET /members, PUT /members/{id}	Admin account management	Member/staff roles and active status
PUT /settings/profile, PUT /settings/password	Account settings	Updates profile or password
POST /notifications/{id}/read	Notification inbox	Marks an item read and follows its action link

Data retrieved

Laravel controllers query Eloquent models for users, plots, plot requests, assignments, calendar events, community updates, and notifications. Responses contain only data appropriate to the authenticated role. Lists support server-side search, status filters, and pagination.

Integration in the website

- Initial page loads return an Inertia response rendered by React.
- `Inertia Link`, `router`, and `useForm` send asynchronous XHR requests for navigation, search, filters, and forms.
- Laravel validates every mutation and returns field errors to the React form without a traditional full-page reload.
- Successful actions redirect back with flash feedback.
- CSRF protection and encrypted Laravel sessions protect all state-changing requests.
- Authorization is enforced on the server even when a button is hidden in the interface.

AJAX/Fetch evidence

The project uses Inertia's XHR transport rather than handwritten

`fetch()` calls. This satisfies the AJAX requirement: searches, filters, pagination, form submissions, notifications, and CRUD actions update through asynchronous browser requests while Laravel remains the source of truth.